RSpec's primitive matchers are similar. They have simple, precise definitions that can't be broken down any further. They're not meant to accept other matchers as input (but you can go the other direction, passing them into other matchers). Typically, they just pass the operation you're performing—an equality check, for example—straight through to the subject of the expectation.

Equality and Identity

RSpec's most fundamental matchers are all concerned with variations of the question, "Are these two things the same?" Depending on context, "the same" might refer to one of several things:

- *Identity*: for example, two references to one object
- *Hash key equality*: two objects of the same type and value, such as two copies of the string "hello"
- *Value equality*: two objects of compatible types with the same meaning, such as the integer 42 and the floating-point number 42.0

Most of the time, Ruby programmers are concerned with the last of these: value equality, embodied in Ruby's == operator.

Value Equality

In RSpec, you use the `eq` matcher to check value equality.

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(Math.sqrt(9)).to eq(3)

# equivalent to:
Math.sqrt(9) == 3
```

Most of the time, this matcher is the one you want. However, sometimes you have a more specific need.

Identity

Say you're testing a `Permutations` class that generates every possible ordering of a set of words. This operation gets expensive quickly, and so you'd like to memoize (cache) the result.

You might initially try using the `eq` matcher to make sure that the second call hits the cached value:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
perms      = Permutations.new
first_try  = perms.of(long_word_list)
second_try = perms.of(long_word_list)

expect(second_try).to eq(first_try)
```